

Cumhuriyet Üniversitesi

İşletim Sistemleri Dersi

Bölüm 4

İş Parçacıkları

(Threads)

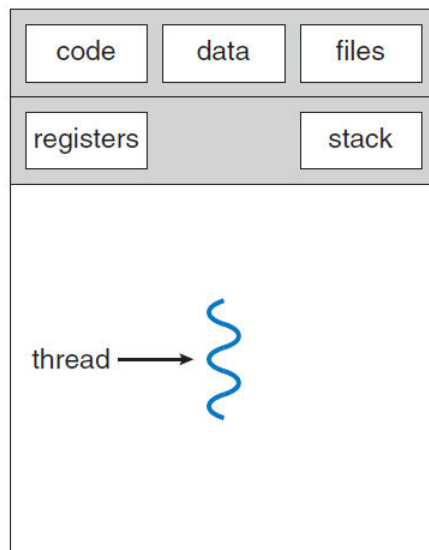
Dr. Halil ARSLAN

Bölümün hedefleri

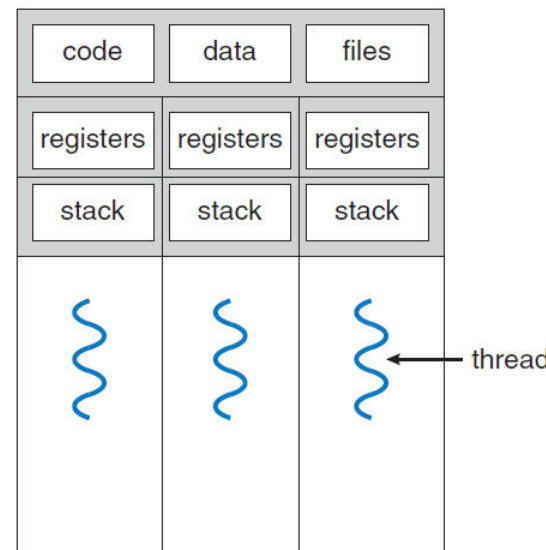
- Thread (iş parçacığı) kavramı
- Multicore programlama
- Multithreading modeller
- Thread kütüphaneleri
- Implicit/dolaylı iş parçacıkları
- Thread konuları
- İşletim sistemi örnekleri

Thread / İş parçacığı

- **Thread**, bir süreç içerisinde, bağımsız olarak yönetilebilen talimatların programlanabilir en küçük yapısı olarak düşünülebilir.
- Geleneksel süreçlerde tek bir thread vardır. Bir süreç içerisinde birden fazla thread kullanılarak birden fazla görevin birlikte yapılması (zaman paylaşımı) sağlanabilir.
- Bir thread, thread id, program sayıcı, kaydedici seti ve yığından oluşur. Böylece bir thread aynı süreç içerisindeki diğer thread'lerle, kod bloğu, veri bloğu ve işletim sistemi kaynaklarını paylaşır.



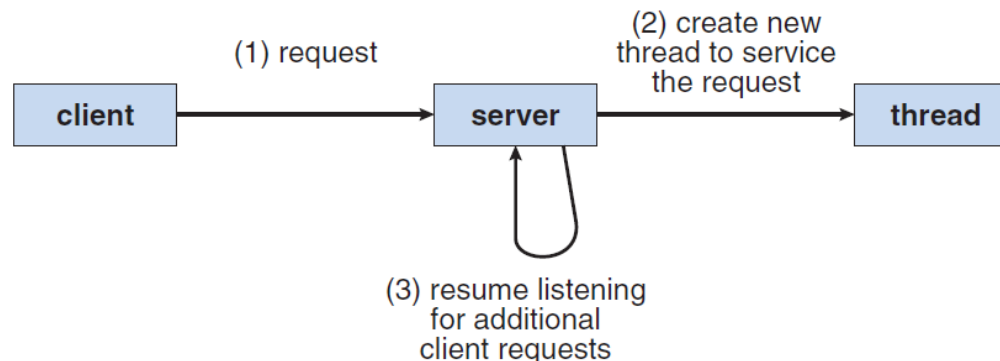
single-threaded process



multithreaded process

Thread

- **Multithreading**, bir süreç içerisinde birlikte yürütülmesi gereken görevlerin her birinin bir thread olarak yer almasıdır.
- Modern uygulamalar çoğunlukla paralel görevler yürütmektedir. Örneğin bir web tarayıcısı bir taraftan ağdan veri transferi yaparken, diğer taraftan web içeriğini görüntülemeyi sürdürmektedir.
- Multithreading tasarlanan uygulamalar, çok çekirdekli sistemlerde işlemci çekirdeklerini paralel kullanarak yoğun işlemci gerektiren görevlerde performanlarını artırabilir.
- Ayrıca bir uygulamanın aynı anda benzer isteklere cevap vermesi gerekebilir. Yoğun bir web sunucusuna aynı anda binlerce erişim isteği gelebilir. Ana süreç istekleri dinlerken, gelen her yeni isteğin cevaplanması için yeni bir thread devreye alınır.

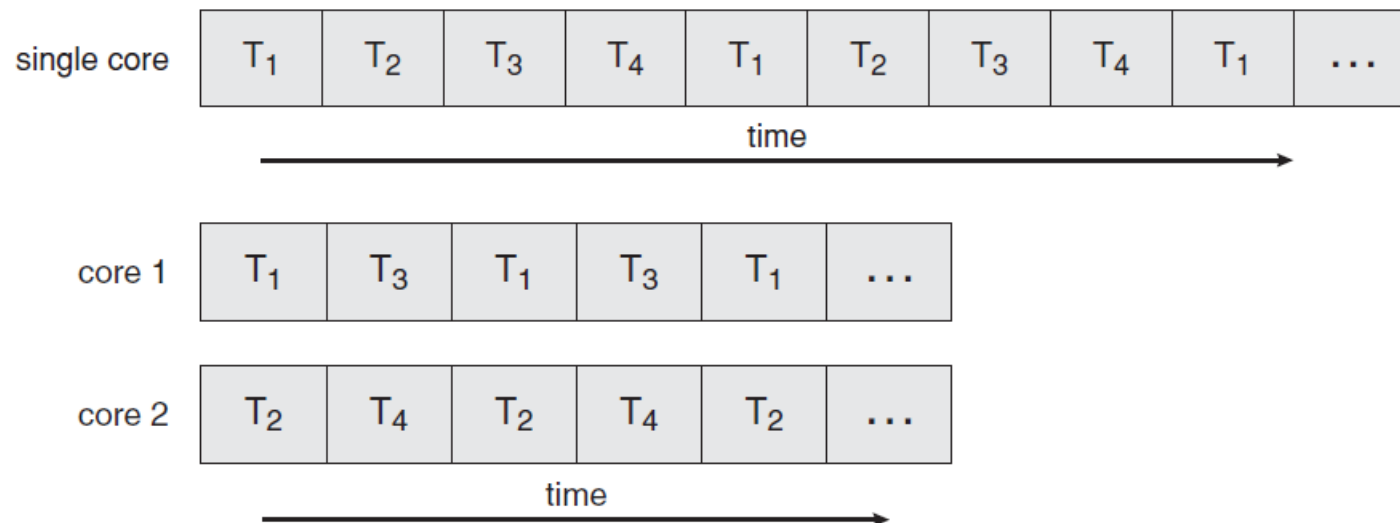


Thread

- Programlamada multithread kullanmanın faydaları;
 1. **Responsiveness (duyarlık):** Programın bir parçasının uzun süreli bir görevi yerine getirirken, uygulamanın tamamının yanıt verememesi istenmez. Bu durumda ilgili görev ayrı bir thread'e alınır.
 2. **Resource sharing (kaynak paylaşımı):** Süreçler arasında iletişim için kullanılan shared memory ve message passing yöntemlerine ihtiyaç duymadan doğası gereği thread'ler süreç içerisinde iletişime sahiptir.
 3. **Economy:** Farklı görevleri yerine getirmek için yeni süreç oluşturmak bellek ve kaynak tahsisi açısından maliyetlidir. Ayrıca süreçler arası geçiş (context-switch) thread'lere göre daha fazla kaynak gerektir.
 4. **Scalability (Ölçeklenebilirlik):** Tek thread'li süreçler işlemci mimarisi kaç çekirdekten oluşursa oluşsun tek işlemci üzerinde çalışırlar. Oysa multithread uygulamalar çok işlemcili sistemlerde farklı çekirdekler üzerinde paralel olarak yürütülebilirler.

Multicore programlama

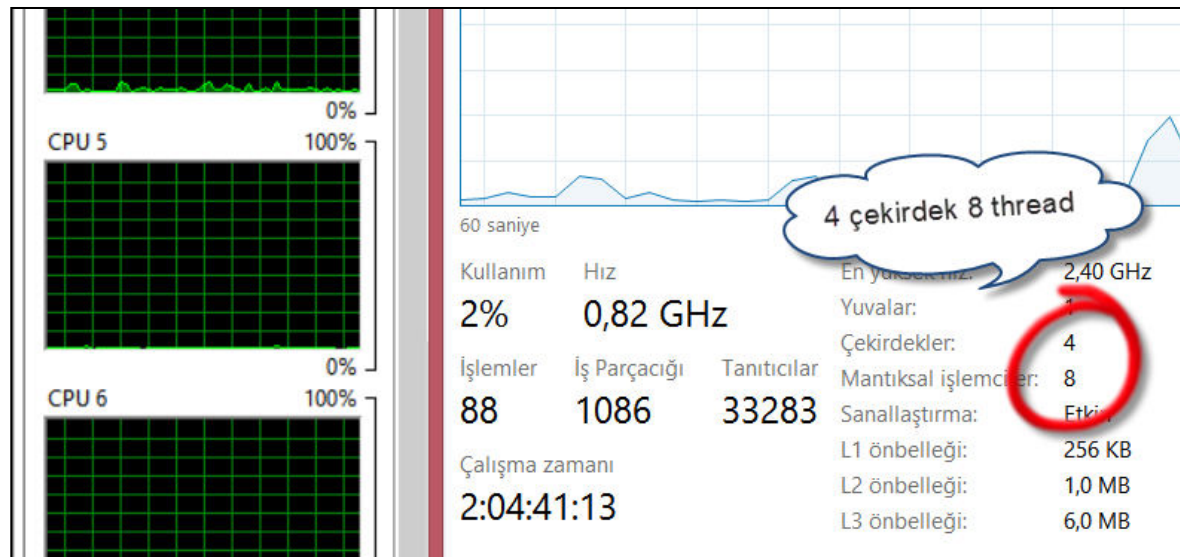
- Başlangıçta bilgisayar sistemlerinin performansını artırmak için tek CPU'lu mimari yerini çoklu-CPU mimarisi geliştirildi.
- Daha sonra tek CPU yongası üzerinde birden fazla çekirdek tercih edildi
- Her bir çekirdek işletim sisteminde ayrı bir CPU olarak görülmektedir.
- Tek yongada çok çekirdekli yada çok işlemcili yapılar **multicore** yada **multiprocessor** sistem olarak isimlendirilir.
- **Multithreaded** programlama ile çok işlemcili mimariler üzerinde eşzamanlı işlem yürütümüyle verimlilik artışı sağlanır.



Multicore programlama

- İki temel kavram, paralellik (**parallelism**) ve eşzamanlılık (**concurrency**) karıştırılmamalıdır.
- **Paralellik**, bir sistemin aynı anda birden fazla görevi yerine getirmesi, **Eşzamanlılık**, tüm görevlerin birlikte yürütülmesine izin verilmesidir.
- Yani bir sistem paralel çalışmasa da eşzamanlı olabilir.
- Modern intel işlemciler çekirdek başına 2 thread yürütümünü destekler. Oracle T4 işlemcileri çekirdek başına 8 thread yürütebilir.

Örnek i7 CPU ile 4 fiziki çekirdek, 8 eşzamanlı thread yürütülür



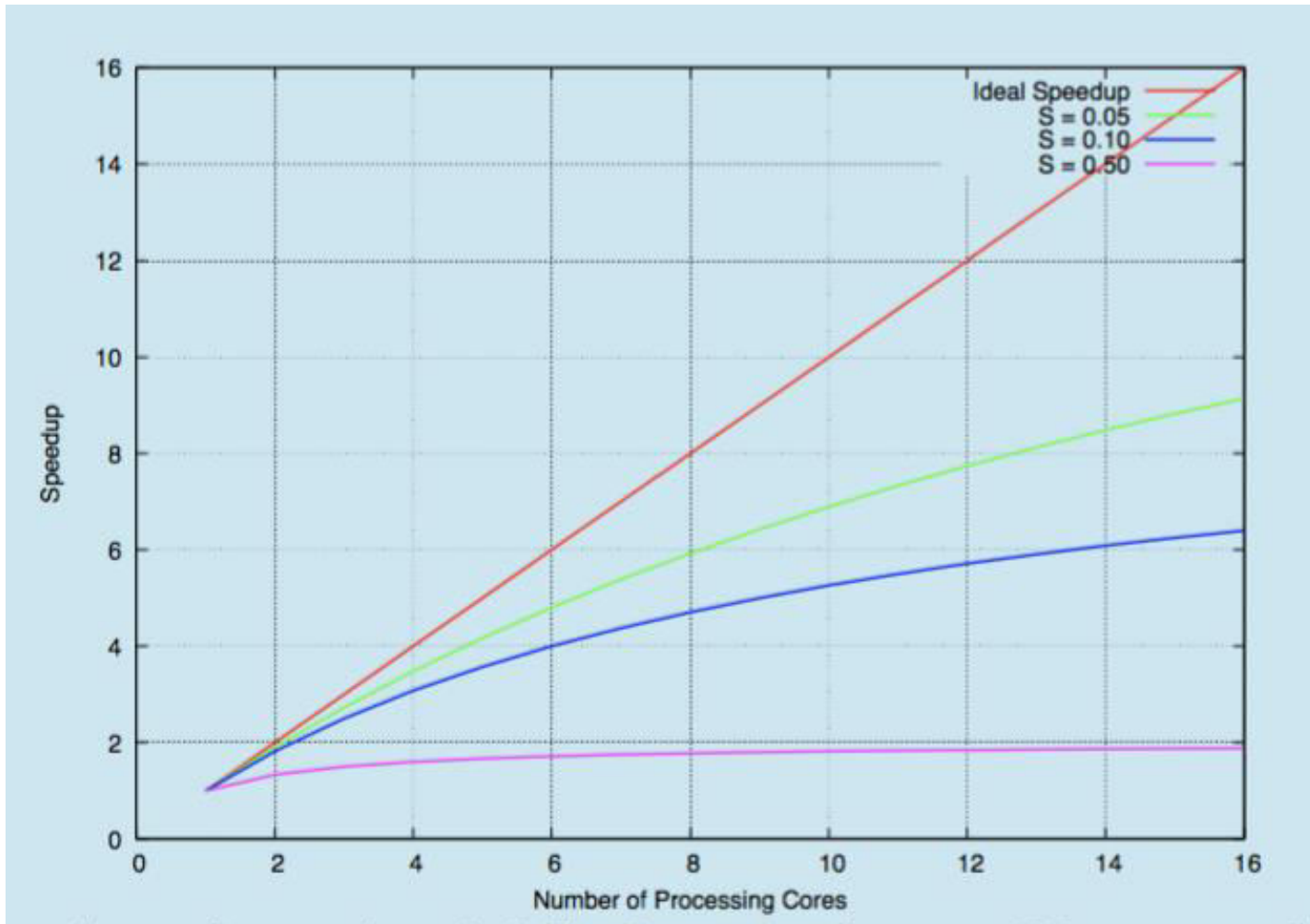
Multicore programlama – Amdahl Kanunu

- **Amdahl kanunu**, paralel ve paralel olmayan bileşenlere sahip bir uygulamanın eklenen her işlemci çekirdeği için performans artışını hesaplar. Formülde S paralel olmayan bileşenlerin oranını, N ise çekirdek sayısını ifade eder.

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- Örneğin, %75'i paralel, %25'i paralel olmayan bileşene sahip bir uygulamanın (S=0,25) 2 çekirdekli bir mimaride (N=2), hız artışı 1,6 kat olacaktır. Çekirdek sayısı 4'e (N=4) çıkarıldığında hız artışı maksimum 2,28 katına çıkacaktır.
- Burada önemli olan husus sistemin çekirdek sayısı ne kadar fazla olursa olsun, uygulamanın paralel olmayan kısmı hız artışını belirlemektedir.
- Örneğin uygulamanın paralel olmayan kısmı %40 ise, işlemci kaç çekirdekli olursa olsun maksimum hız artışı 2,5 kat olmaktadır.

Multicore programlama – Amdahl Kanunu



Multicore programlama - Zorlukları

- Multicore sistemlerdeki gelişmeler programcıların yazılımlarını multithread yapıda tasarlamasını teşvik etmektedir. Bu tasarımlarda 5 temel konu göz önüne alınmalıdır.
- 1. **Identifying tasks** (görev tanımı): Uygulamada eşzamanlı çalışabilecek kısımların bulunarak paralel olarak çalışabilecek birbirinden bağımsız görevlerin tanımlanmasıdır.
- 2. **Balance** (denge): Paralel olarak çalışacak görevler belirlenirken, görevlerin eşit iş yüklerine sahip olmasına dikkat edilmelidir.
- 3. **Data splitting** (veri bölme): Görevler ayrıştırılırken, bu görevlerin işlediği veri setleri de ayrıştırılmalıdır.
- 4. **Data dependency** (veri bağımlılığı): Görevler arasında işlenen veri setleri senkronizasyon açısından değerlendirilmelidir. Bir görev başka bir görevin sunduğu veri setini işleyecekse senkronizasyon gerektirir.
- 5. **Testing, debugging**: Multicore çalışan uygulamaların testi ve hata ayıklaması tek thread yapıdaki uygulamalara göre daha zordur.

Multicore programlama – Türleri

- İki tür paralelleştirme yaklaşımı vardır. Bunlar **data parallelism** ve **task parallelism**'dir.
- **Data parallelism**, veri paralelleştirme yaklaşımında aynı işlemi gerçekleştiren görevler arasında verilerin paylaşılmasına dayanır.

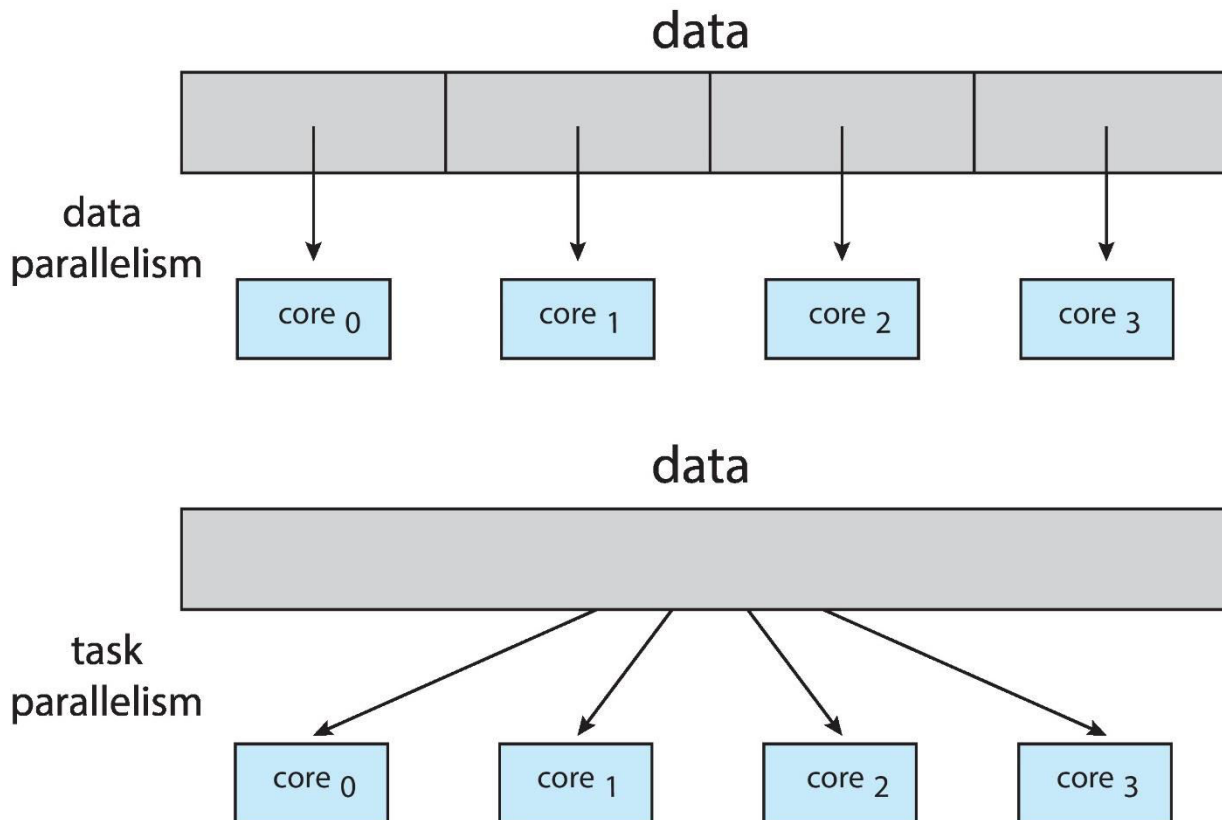
Örneğin N elemanlı bir dizinin elemanlarının toplanması görevinde birinci çekirdek $[0 .. (n/2)-1]$ arası elemanları işlerken diğer çekirdek $[n/2 .. n-1]$ arası elemanları işler.

- **Task parallelism**, görev paralelleştirme yaklaşımında farklı çekirdeklerde farklı görevlerin yürütümü planlanır. Her thread farklı bir operasyonu gerçekleştirir. Thread'ler aynı veri setlerini kullanabileceği gibi farklı veri setlerini de kullanabilir.

Örneğin N elemanlı bir dizi içerisinde birinci thread istatistiksel bir işlem yapmak üzere programlanırken diğer thread aynı veri seti üzerinde başka bir istatistiksel hesaplama yapmak üzere programlanabilir. Böylece aynı veri setini kullanan thread'ler farklı işlemci çekirdeklerini kullanabilirler.

Temelde, çoğu uygulama bu iki yaklaşımı birlikte kullanır.

Multicore programlama

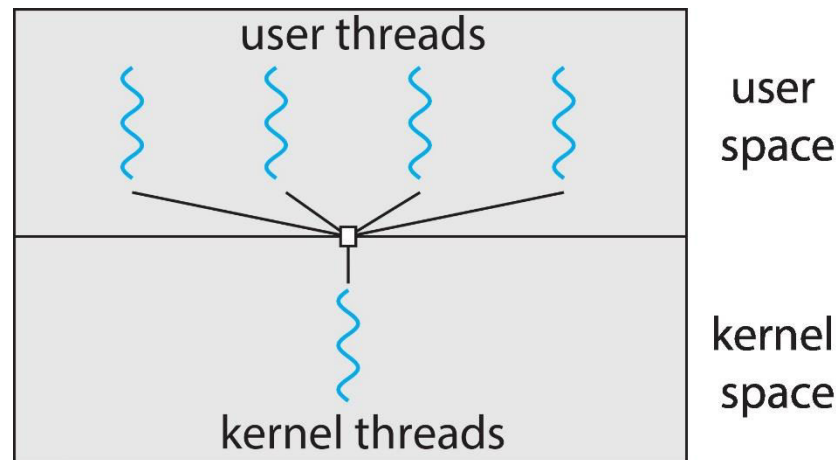


Multithreading modeller

- Modern işletim sistemlerinde thread'ler, kullanıcı seviyesinde **user thread**, sistem çekirdeği seviyesinde **kernel thread** ile tanımlanabilir.
- **User thread'ler**, kernel desteği olmaksızın planlama, oluşturma ve yönetim gibi işlemler user-space'de yapılır ve kullanıcı kütüphaneleri (pthreads, c-threads) tarafından türetilirler. Çok işlemcili yapılarda paralel çalışma garantileri yoktur.
- **Kernel thread'ler**, kernel-space'de tanımlanarak doğrudan işletim sistemi tarafından yönetilirler ve user thread'ler için kernel erişimleri sağlarlar. Platforma özgüdürler ve user thread'lere göre daha maliyetlidir.
- Tüm modern işletim sistemleri (windows, mac, linux) kernel thread'lere destek verirler.
- User thread'lerle kernel thread'ler arasında bir ilişki olmalıdır. Bu ilişki **many-to-one**, **one-to-one** ve **many-to-many** model olabilir.

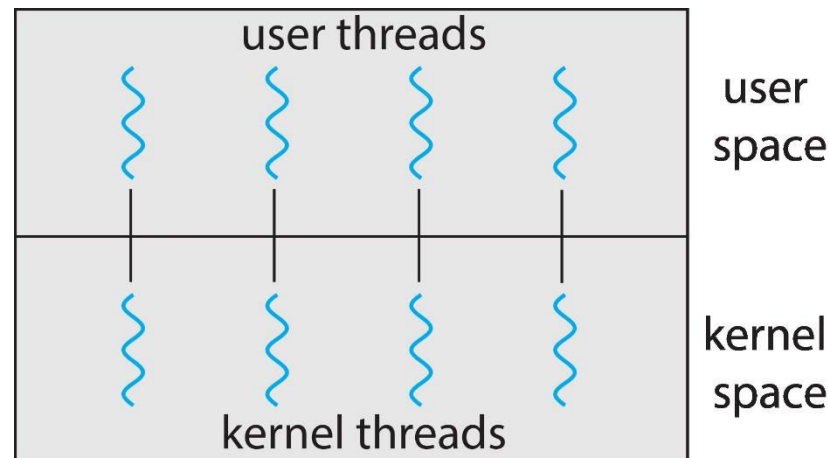
Multithreading modeller/many-to-one

- Bu modelde tek bir kernel thread'e, çok user thread bağlanır. Thread yönetimi kullanıcı kütüphaneleri üzerinden yapılır. Verimli olmakla birlikte bir sistem çağrısı bir thread'i bloklarsa tüm süreç bloklanacaktır.
- Multicore sistemlerde yalnızca aynı anda bir thread çekirdeğe erişebildiğinden uygulamanın paralel olarak çalışması mümkün değildir.
- Java'nın eski sürümlerinde olan bu model multicore sistemlerin gelişmesiyle etkinliğini yitirmiştir.
- Solaris Green Threads, GNU Portable Threads



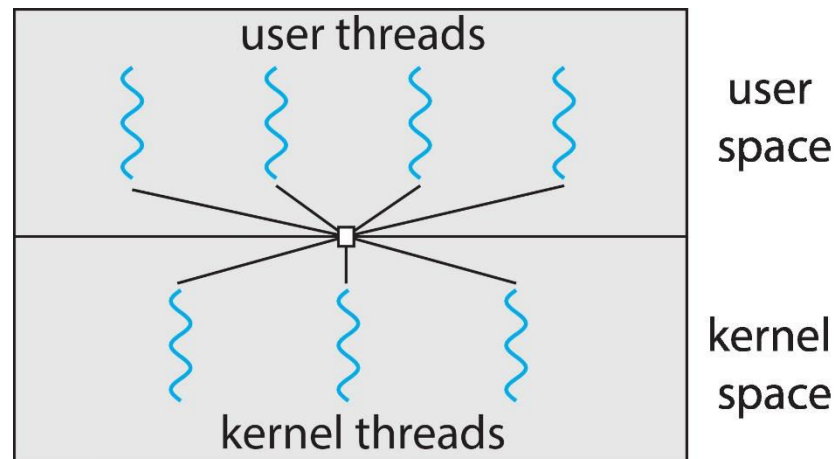
Multithreading modeller/one-to-one

- Bu modelde her bir user thread, bir kernel thread'e bağlanır.
- Bir thread sistem çağrısı ile bloklandığında diğer thread'ler sürdürülmeye devam eder.
- Uygulamanın multicore sistemlerde paralel çalışmasına olanak sağlanır.
- Ancak her bir user thread'in bir kernel thread üretmesi gereksinimi sistemin kernel thread sınırıyla kısıtlanabilir ve ayrıca kernel thread üretimi performansı azaltabilir.
- Günümüz Linux ve Windows işletim sistemleri bu modeli kullanmaktadır.



Multithreading modeller/many-to-many

- Bu modelde tüm user thread'ler, aynı sayıda yada daha az sayıda kernel thread'e bağlanır. Bağlanılan kernel thread sayısı uygulamaya yada makineye özgü olabilir.
- Daha önce sunulan iki modelin dezavantajlarını gidermek için geliştirilmiştir. Birinci modelde thread tanımlama hızlıdır ancak paralelleştirme mümkün değildir. İkinci modelde ise sistem thread sınırı aşılabilmektedir. (i7-64 8 GB ram için aynı instance'da 2000 civarında sorun yok)
- Windows, Fibers paketi ile birlikte kullanılır.



Thread kütüphaneleri

- Thread kütüphaneleri, programcıya thread oluşturmak ve yönetmek için API sunar. Bir thread kütüphanesini kullanmanın iki yolu vardır.
 - Birinci yaklaşımda, hiçbir çekirdek desteği olmadan thread'lerin kullanıcı uzayındaki bir kütüphane üzerinden yürütülmesidir.
 - İkinci yaklaşımda ise doğrudan işletim sistemi tarafından desteklenen çekirdek düzeyinde bir kütüphaneden uygulamaktır.
- Günümüzde 3 temel thread kütüphanesi: Pthreads, Windows ve Java'dır
- **Pthreads**, POSIX sistemlerin standart thread kütüphanesidir ve hem kullanıcı düzeyinde hem de kernel düzeyinde erişim sağlanabilir.
- **Windows**, windows sistemler için çekirdek düzeyinde kullanılır.
- **Java**, doğrudan java programları içinden thread tanımlama ve yönetmek için kullanılan kütüphanedir. Ancak JVM bir işletim sistemi üzerinde çalıştığı için sistem düzeyinde windows kütüphanesine yada pthread kütüphanesine bağlanır.

Thread kütüphaneleri

- Multithread uygulamalarda iki tip yaklaşım vardır.
- **Asynchronous threading:** Parent thread bir child thread oluşturur ve iki thread'de eşzamanlı olarak çalışmasını sürdürür.

Her thread bağımsız olarak çalışır ve parent thread child thread'in sonlandığını bilmez. Genellikle veri paylaşımı gerektirmeyen bağımsız görevlerde tercih edilir.

- **Synchronous threading:** Parent thread bir veya daha fazla child thread oluşturur ve parent thread görevine devam etmeden önce tüm child thread'lerinin sonlanmasını bekler.

Genellikle ilişkili verilerin child thread'lerde işlendikten sonra parent thread'de sonuçların birleştirilmesi gibi senaryolarda uygulanır.

Thread kütüphaneleri - Pthreads

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
int sum; /* thread'lerde paylaşılan veri */
void *runner(void *param); /* thread'in çağırdığı fonksiyon */
int main(int argc, char *argv[]){
    pthread_t tid; /* thread kimliği */
    pthread_attr_t attr; /* thread özellikleri */
    if (argc != 2) {
        fprintf(stderr, "Kullanım hatası: thread.out <integer value>\n");
        return -1;
    }
    if (atoi(argv[1]) < 0) {
        fprintf(stderr, "%d >= 0 olmalı\n", atoi(argv[1]));
        return -1;
    }
    pthread_attr_init(&attr); /*thread özelliklerini ata - varsayılan*/
    pthread_create(&tid, &attr, runner, argv[1]); /* thread oluştur */
    pthread_join(tid, NULL); /* thread bitene kadar bekle */
    printf("sum = %d\n", sum);
}

void *runner(void *param){
    int i, upper = atoi(param);
    sum = 0;
    for (i = 1; i <= upper; i++)
        sum += i;
    pthread_exit(0);
}
```

```
$ gcc -pthread -o
thread thread.c
```

Thread kütüphaneleri - Windows

```
#include <windows.h>
#include <stdio.h>
int Toplam; /* Hesaplanacak global degisken */
int WINAPI topla(int *Param){
    int i;
    for ( i = 0; i <= *Param; i++) Toplam += i;
    return 0;
}
int main(int argc, char *argv[]){
    int ThreadId, Param;
    HANDLE ThreadHandle;
    if (argc != 2){
        fprintf(stderr,"Tamsayi parametre gerekli\n");
        return -1;
    }
    Param = atoi(argv[1]);
    if (Param < 0){
        fprintf(stderr,"Pozitif tamsayi gir\n");
        return -1;
    }
    ThreadHandle = CreateThread(NULL, /* güvenlik özellikleri */
                                0,    /* stack boyutu */
                                topla, /* thread fonksiyonu */
                                &Param, /* thread parametresi */
                                0,    /* tanımlama bayrağı */
                                &ThreadId); /* thread id */
    if (ThreadHandle != NULL) {
        WaitForSingleObject(ThreadHandle, INFINITE); /* bekle */
        CloseHandle(ThreadHandle);
        printf("Sonuc = %d\n", Toplam);
    }
}
```

Thread kütüphaneleri - Java

- Bir java programının çalıştırılmasının temeli thread'lere dayanmaktadır.
- Sadece main() fonksiyonu içeren bir java programı dahi bir thread'den oluşmaktadır.
- Java API'yı thread tanımlamak ve yönetmek için zengin içerik sağlar
- JVM yüklü her sistemde java thread'leri kullanılabilir.
- Java thread'leri tanımlamak için 2 teknik vardır.
- Birinci yaklaşımda Thread sınıfından yeni bir sınıf tanımlanır ve run() metdodu override edilir.
- İkinci yaklaşımda ki yaygın kullanımdır, Runnable interface'den implemente edilen bir sınıf tanımlanır.

```
public interface Runnable
{
    public abstract void run();
}
```

Thread kütüphaneleri - Java

```

class Sum {
    private int sum;
    public int getSum() {
        return sum;
    }

    public void setSum(int sum) {
        this.sum = sum;
    }
}

class Summation implements Runnable {
    private int upper;
    private Sum sumValue;
    public Summation(int upper, Sum sumValue) {
        this.upper = upper;
        this.sumValue = sumValue;
    }

    public void run() {
        int sum = 0;
        for (int i = 0; i <= upper; i++)
            sum += i;
        sumValue.setSum(sum);
    }
}

```

```

public class ThreadDemo {

    public static void main(String[] args) {
        if (args.length > 0) {
            if (Integer.parseInt(args[0]) < 0){
                System.err.println("parametre gerekli");
            } else {
                Sum sumObject = new Sum();
                int upper = Integer.parseInt(args[0]);
                Thread thrd = new Thread(new Summation(upper, sumObject));
                thrd.start();
                try {
                    thrd.join();
                    System.out.println("Toplam:" + sumObject.getSum());
                } catch (InterruptedException ie) {
                }
            }
        } else{
            System.err.println("parametre gerekli");
        }
    }
}

```

Implicit/Dolaylı Thread

- Multicore işlemcilerdeki gelişmeler, uygulamaların yüzlerce thread içermesine yol açmaktadır.
- Ancak bu tür uygulamaları geliştirmek oldukça zahmetlidir.
- Günümüzde giderek, derleyiciler çalışma zamanında uygulamaların thread yönetimlerini ele almaktadır. Bu yaklaşım **implicit thread** olarak ifade edilmektedir.
- Derleyiciler dışında thread pool, OpenMP ve Grand Central Dispatch olarak bilinen 3 alternatif yaklaşım ele alınacaktır.

Implicit Thread - Thread Pool

```
public class WorkerThread implements Runnable {

    private String command;
    public WorkerThread(String s) {
        this.command=s;
    }

    @Override
    public void run() {

        System.out.println(Thread.currentThread().getName()+"
        Basladı = " + command);
        processCommand();

        System.out.println(Thread.currentThread().getName()+"
        Bitti.");
    }

    private void processCommand() {
        try {
            Thread.sleep(5000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    @Override
    public String toString(){
        return this.command;
    }
}
```

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ThreadPool {

    public static void main(String[] args) {
        ExecutorService executor =
        Executors.newFixedThreadPool(5);
        for (int i = 0; i < 10; i++) {
            Runnable worker = new
            WorkerThread(""+ i);
            executor.execute(worker);
        }
        executor.shutdown();
        while (!executor.isTerminated()) {
        }
        System.out.println("Tum thread'ler
        bitti");
    }
}
```


Implicit Thread - OpenMP

- OpenMP, shared-memory ortamlarında, paralel programlama için C, C++ ve Fortran dillerinde yazılmış programlar için geliştirilmiş bir API'dır.
- OpenMP ile paralel olarak çalıştırılacak kod bölümleri tanımlanır.
- Paralel olarak çalıştırılacak kod blokları belirlendikten sonra bu bloklar derleyici direktifi olarak OpenMP kütüphanesi tarafından çalışma zamanında paralel olarak yürütülür.
- `#pragma omp parallel` bloğu paralel çalıştırılacak kod bloğunu belirler.
- `#pragma omp parallel for` bloğu paralel çalıştırılacak döngüyü ifade eder.
- OpenMP'nin Windows, Linux ve Mac Osx gibi sistemler için ticari ve açık kaynak kütüphane olarak dağıtımları bulunabilir.
- Windows üzerinde C::B için TDM-GCC (<http://tdm-gcc.tdragon.net/>) kütüphanesi kullanılabilir.

Implicit Thread - OpenMP

- Aşağıdaki kod main thread altında çalışan yeni fork thread'ler tanımlar
- Her thread paralel olarak koşturulur ve kendi thread id'sini ekrana basar. İlgili makinede çalışabilecek paralel thread sayısını tanımlar.
- Main thread son olarak toplam thread sayısını verecektir.

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main (int argc, char *argv[]){
    int nthreads, tid;

    #pragma omp parallel private(nthreads, tid)
    {
        tid = omp_get_thread_num();
        printf("calisan thread = %d\n", tid);
        if (tid == 0){
            nthreads = omp_get_num_threads();
            printf("Toplam thread = %d\n", nthreads);
        }
    }
}
```

Derleyici parametresi
Other Options

–fopenmp

linker kütüphanesi
→ libgomp.a

Implicit Thread - GDC

- Grand Central Dispatch – GDC, Apple’ın Mac OS X ve iOS işletim sistemleri için C ve C++ dillerinde paralel çalıştırılacak kod bloklarını tanımlar.
- OpenMP’ye benzer şekilde paralel çalıştırılacak kod blokları için çalışma zamanı kütüphanesidir.
- `^{ printf("I am a block"); }` karakterinden sonraki blok `{ }` arasındaki bölüm paralel olarak yürütülecektir.
- Thread Pool, OpenMP ve GDC multithreading uygulamaları yürütmek için geliştirilen birkaç teknolojiden sadece bazılarıdır.
- Bunların dışında pekçok ticari ve açık kaynaklı teknolojiler bulunmaktadır.
- Bunlardan `java.util.concurrent`, PFX, MPI incelenebilir.

Thread Konuları

fork() ve exec():

- Bir thread fork() sistem çağrısı ile child süreç tanımlarsa, yeni süreç parent süreçteki tüm thread'leri kopyalayabilir yada yeni süreç tek thread'den oluşabilir.
- Eğer fork() çağrısından sonra exec() çağrısı ile ayrı bir süreç çağrılacaksa, parent sürecin tüm thread'lerinin kopyalanması gereksizdir.
- Ancak fork() çağrısından sonra ayrı bir süreç yürütülmeden parent süreç içeriğinden devam edilecekse tüm thread'lerin kopyalanması gerekebilir.
- Bu durum sisteme bağlı olarak tasarlanabilir.

Thread Konuları

Signal yönetimi:

- UNIX sistemlerde **signal** bir olayın (event) meydana geldiğini tanımlar.
- Sinyaller, olayın nedenine ve kaynağına bağlı olarak senkron yada asenkron alınabilir.
- Senkron sinyallere sıfıra bölme hatası, izinsiz bellek erişimi örnek gösterilebilir. Senkron sinyallar, olayın oluştuğu sürece iletilen sinyallerdir.
- Asenkron sinyaller, farklı süreçlerden alınan sinyallerdir. Örneğin süreç yürürken CTRL+C ile sürecin sonlandırılması.

Thread Konuları

Signal yönetimi:

- Multithread yapılarda sinyaller;
- (1)Sinyal, sinyalin uygulandığı thread'e dağıtılır, (2)Sinyal, süreçteki tüm thread'lere dağıtılır, (3)Sinyal, süreçteki bazı thread'lere dağıtılır, (4)Tüm sinyaller, süreçteki özel bir thread'e atanmış olabilir. Bu durum sinyalin durumuna bağlıdır.
- Örn. CTRL+C tüm thread'lere dağıtılır. Yine senkron sinyaller sadece ilgili thread'e dağıtılır.
- Unix sistemlerde bir sinyalin ilgili thread'e iletimi aşadaki gibidir.

```
kill(pid_t pid, int signal)
```

Thread Konuları

Thread iptali:

- Multithread yapılarda yürüyen bir thread tamamlanmadan iptal edilmesi gerekebilir.
- Örneğin çoklu thread yapısında bir veritabanında arama yapılırken, aranan içeriğin bulunması sonucundan diğer threadlerin iptali gerekir.
- (1) Bir thread başka bir thread tarafından doğrudan iptal edilebilir (**asynchronous**) yada (2) kendini iptal etmesi istenebilir (**deferred**).
- `pthread_cancel(tid)`; isteği thread'i iptal eder, ancak hangi modda iptalin gerçekleşeceği thread tarafından işlenir.
- İlgili thread iptal modu olarak; iptal edilemez **off**, **asynchronous** yada **deferred** modda yapılandırılmış olabilir.

Thread Konuları

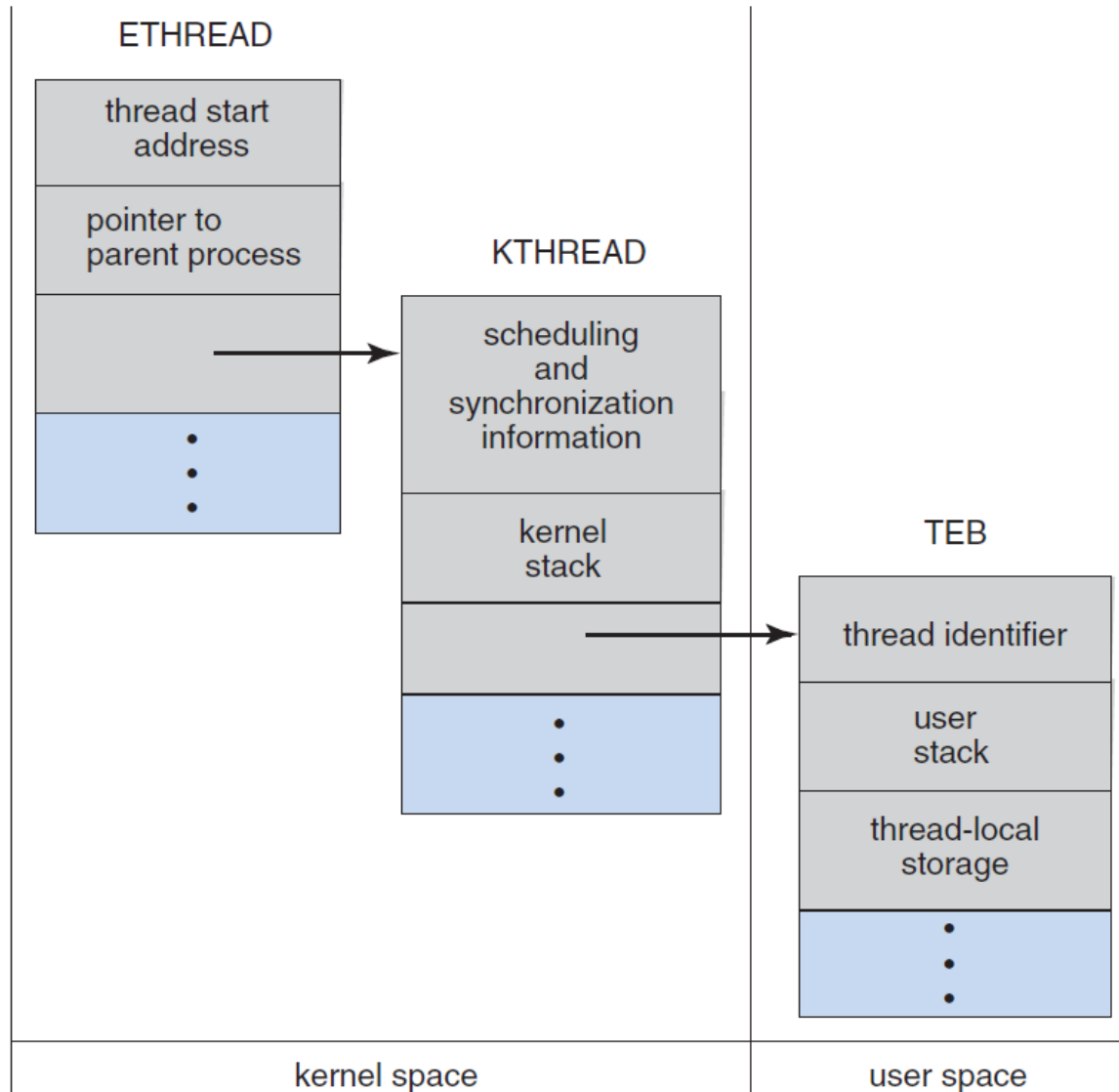
Yerel depolama:

- Thread'ler kendinelerine ait yerel depolama birimleri kullanabilir.
- Thread-Local Storage (TLS) olarak isimlendirilen bu yapılar, yerel değişkenlerle karıştırılmamalıdır.
- Yerel değişkenler sadece ilgili fonksiyonda geçerliyen, TLS'ler tüm thread boyunca kullanılır.
- Posix, windows ve java thread kütüphaneleri TLS'leri destekler

İşletim Sistemleri - Windows

- Windows uygulamaları bir veya daha fazla sayıda thread içeren ve one-to-one modelde kullanıcı thread'lerinin kernel thread'lerine haritalandığı şekilde yürütülürler.
- Windows thread'lerinin genel içeriği; (1) Thread kimliğini belirten ID, (2) İşlem durumunu tanımlayan bir register kümesi, (3) kullanıcı ve kernel modlarına ait user-stack ve kernel-stack, (4) dll'ler ve çalışma zamanı kütüphaneleri için özel depolama alanlarıdır.
- Register'lar, stack'ler ve özel depolama birimleri thread içeriğini ifade eder. Bir thread'in özel veri yapısı;
 - **ETHREAD**: Yürütülen thread bloğu - Ait olduğu süreç ve yürütülen rutin adresinin yanında, kernel thread için pointer içerir. (kernel space'dedir)
 - **KTHREAD**: Kernel thread bloğu - Thread'in planlama ve senkronizasyon bilgisini, kernel stack'ini ve TEB pointer'ini içerir. (kernel space'dedir)
 - **TEB**: Thread çevresel ortam bloğu - user-space'de thread'lerin eriştiği veri modelidir ve Thread ID, kullanıcı stack'i ve thread yerel depolarını içerir.

İşletim Sistemleri - Windows



İşletim Sistemleri - Linux

- Linux geleneksel olarak **fork()** sistem çağrısı ile süreç çoğaltırken **clone()** sistem çağrısı ile de yeni thread'ler tanımlar.
- Linux'de süreç-thread ayrımı yoktur ve ikisi de görev (task) terimini kullanır.
- **clone()** çağrısı çağrıldığında, parent ve child task'lar arasında nelerin paylaşılabileceği parametresi geçer.
- Parent ve child task'lar aynı dosya sistemini (çalışma dizini), aynı bellek alanını, aynı sinyalleri ve açık dosyaları kullanabilir.

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.

JVM ve Konak İşletim Sistemi

- JVM genellikle bir konak işletim sistemi üzerine uygulanır.
- Bu yaklaşım, JVM destekleyen konak işletim sisteminin ayrıntılarının gizlenerek, java programları için sanal bir çalışma ortamı sunulmasını sağlar.
- JVM, Java thread'lerinin işletim sistemine özgü haritalanması yerine bu haritalamayı uygulamaya ve JVM platformuna özgü yapar.
- Örneğin Windows XP one-to-one modeli kullanır ve JVM bu platformda her java thread'ini bir kernel thread'le haritalar. Benzer şekilde UNIX'de many-to-many modeline göre haritalar.
- Ayrıca Java thread kütüphaneleri ile konak işletim sistemi kütüphaneleri arasında ilişki kurulabilir.
- Örneğin Java, Windows ailesi işletim sistemleri için Windows API'sini, Linux, Solaris ve Mac OS X işletim sistemleri için Pthread API'sini kullanabilir.